38. Linux on Cloud VMs: metadata, identity, disks

38. Linux on Cloud VMs: metadata, identity, disks

Phase: Phase 11 — Containers and cloud Linux
Estimated time: 1–2 days
Slug: `38-cloud-vms`

Learning objectives

- Use instance metadata thoughtfully
- Attach/mount data disks
- Prefer IAM roles over long-lived keys
- Automate with cloud-init basics

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

On a cloud provider you do not install an operating system from an ISO. You launch a **VM from an image**, and it must configure itself on first boot: set the hostname, create your user, install your SSH key, install packages, and run your setup commands. The standard tool for that on Linux is **cloud-init**. Understanding it lets you create repeatable servers, debug "my key was not installed", and avoid hand-configured snowflakes.

Core concepts

Term	Meaning
image (AMI, qcow2, …)	Disk template the VM boots from
instance type	CPU/RAM size of the VM
metadata service	Link-local HTTP endpoint at `169.254.169.254` giving instance info
user-data	Your script or cloud-config passed at launch
cloud-init	Boot-time agent that reads metadata and user-data and configures the VM
security group	Cloud firewall around the instance
key pair	SSH public key injected at first boot
ephemeral disk	Local storage lost when the instance stops/terminates

The purpose of cloud-init: **first-boot configuration** of a generic image from metadata and user-data.

```
 launch request (image, type, key, user-data)
        |
        v
   hypervisor boots VM --> kernel --> systemd --> cloud-init stages:
        init-local -> init (network, metadata) -> config -> final (runcmd)
                                        |
                     reads http://169.254.169.254/ (metadata + user-data)
        |
        v
  hostname set, users + keys created, packages installed, scripts run
```

Stages: `cloud-init-local`, `cloud-init`, `cloud-config`, and `cloud-final`. Most modules run once per instance, some every boot.

Worked examples

A cloud-config user-data file (must start with `#cloud-config`):

```yaml
#cloud-config
hostname: web-01
users:
  - name: deploy
    groups: [sudo]
    shell: /bin/bash
    sudo: "ALL=(ALL) NOPASSWD:ALL"
    ssh_authorized_keys:
      - ssh-ed25519 AAAA... you@laptop
package_update: true
packages: [nginx, fail2ban]
write_files:
  - path: /etc/motd
    content: |
      managed by cloud-init
runcmd:
  - systemctl enable --now nginx
  - [sh, -c, "echo ready > /var/tmp/ready"]
```

Validate before launch:

```
cloud-init schema --config-file user-data.yaml
```

Launch examples (provider CLIs differ; shown for shape only):

```
aws ec2 run-instances --image-id AMI --instance-type t3.micro \
  --key-name mykey --user-data file://user-data.yaml
```

Check what happened on the VM:

```
cloud-init status --long
sudo cat /var/log/cloud-init-output.log | tail -n 30
```

```
sudo less /var/log/cloud-init.log
sudo cloud-init analyze show
```

Read instance metadata (do this from inside the VM):

```
curl -s http://169.254.169.254/latest/meta-data/instance-id    # AWS IMDSv1 style
TOKEN=$(curl -sX PUT http://169.254.169.254/latest/api/token \
  -H 'X-aws-ec2-metadata-token-ttl-seconds: 60')
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/local-ipv4
cloud-init query ds.meta_data
```

Test locally without a cloud, using a container or multipass:

```
multipass launch 24.04 --name ci-test --cloud-init user-data.yaml
multipass exec ci-test -- cloud-init status --wait
```

Re-run on a lab VM (never on production):

```
sudo cloud-init clean --logs
sudo reboot
```

Basic cloud hygiene:

```
lsblk
df -h
ip -br addr
sudo journalctl -b -u cloud-init -u cloud-final --no-pager | tail
```

Mini-lab

1. Write a cloud-config that creates a user, installs one package, and writes a file with `write_files`.
2. Launch a VM (free tier, multipass, or a local libvirt image) with it.
3. SSH in and run `cloud-init status --long`. Verify the file and package.
4. Introduce a YAML typo, relaunch, and find the error in `/var/log/cloud-init.log`.
5. From the VM, query metadata for the instance id and private IP.
6. Terminate the VM and write down what was lost (ephemeral disk) and what you can recreate from the user-data alone.

Decision guide / failure modes

Need	Choose
Per-instance first-boot setup	cloud-init user-data
Ongoing configuration changes	Ansible or similar, not re-running cloud-init
Same server many times	custom image (Packer) + small user-data
Secrets	secret manager / instance role, not user-data
Debug boot	`cloud-init status`, console log, serial console

Failure modes:

- First line must be `#cloud-config` or `#!/bin/bash`; otherwise it is ignored.
- User-data is visible to anyone who can read metadata. No secrets there.
- Scripts that need network but run too early; use `runcmd` (final stage).
- Instance with no ingress rule in the security group looks “broken” but is just firewalled.
- Wrong login user: images use `ubuntu`, `ec2-user`, `admin`, or `debian`.
- Treating VMs as pets. Anything done by hand disappears when you rebuild.

Common mistakes

- IMDSv1 on exposed hosts
- Secrets baked into AMIs
- Data on ephemeral disks only

Exercises

Work in `exercises/38-cloud-vms/`. Read that folder’s `README.md` fully before coding.

```
cd exercises/38-cloud-vms
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement `cloud_init_purpose` in `exercise.sh` so it prints a short hyphenated phrase that states what cloud-init is for: configuring a generic cloud image the first time an instance boots. Use the lowercase hyphenated form from the lesson. Afterwards, write a small cloud-config on paper or in a file that creates one user and installs one package, and explain which boot stage runs each part. Keep the function body to one line; the goal is recall of the lesson vocabulary, not cleverness. Finish by re-reading the Decision guide table and noting one failure mode you could have caused yourself.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/38-cloud-vms/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. What is cloud-init?
2. Why IAM roles over keys?
3. Metadata risk?

Further reading

- https://cloudinit.readthedocs.io/